

Proyecto Final: Sistema de Mapeo Cartográfico e Interacción Vectorial Isótropa Local

Materia: Graficación por Computadora

Desarrollador: Edgarsc0

Repositorio: [Edgarsc0/proyecto_computer_graphics](https://github.com/Edgarsc0/proyecto_computer_graphics)

14 de junio de 2026

Índice

1. Introducción	4
1.1. Contexto general	4
1.2. Importancia de la graficación en el problema	4
1.3. Descripción breve del proyecto	4
2. Planteamiento del problema	4
2.1. Descripción detallada del problema	4
2.2. Situación actual	5
2.3. Justificación	5
2.4. Alcances y limitaciones	5
3. Objetivos	5
3.1. Objetivo general	5
3.2. Objetivos específicos	5
4. Marco teórico	6
4.1. Conceptos fundamentales de graficación por computadora	6
4.2. Técnicas utilizadas	6
4.2.1. Transformaciones geométricas afines 2D	6
4.2.2. Pipeline gráfico del navegador	6
4.2.3. Modelo de Iluminación y Estilos Visuales Premium	7
4.3. Tecnologías relacionadas	7
5. Propuesta de solución	7
5.1. Descripción general de la solución	7
5.2. Enfoque adoptado	7
5.3. Justificación técnica de la propuesta	8
6. Desarrollo del proyecto	9
6.1. Análisis	9
6.1.1. Requerimientos funcionales	9
6.1.2. Requerimientos no funcionales	9
6.2. Diseño	9
6.2.1. Arquitectura del sistema	9
6.2.2. Diagrama de flujo de coordenadas	10
6.3. Construcción (Implementación)	10
6.3.1. Herramientas utilizadas	10
6.3.2. Algoritmos gráficos y proyección matemática empleados	11
6.4. Pruebas	12
6.5. Implantación	12
7. Resultados	13
8. Discusión	13
8.1. Ventajas de la solución	13
8.2. Limitaciones y problemas encontrados	13

9. Conclusiones	14
10.Referencias	15
11.Anexos	16
11.1. Código de Proyección Isótropa Geodésica (Frontend)	16
11.2. Código de Soporte Gestual Pinch-to-zoom	16

1 Introducción

1.1 Contexto general

El mapeo de espacios geográficos a nivel local ha tomado gran importancia con el desarrollo de la web móvil y la geolocalización. Las plataformas tradicionales de Sistemas de Información Geográfica (GIS) suelen ser pesadas, costosas y difíciles de personalizar para necesidades específicas en pantallas web de alta interactividad. En este escenario, la graficación vectorial 2D mediante el estándar SVG (Scalable Vector Graphics) ofrece una alternativa ligera y potente capaz de renderizar geometrías arbitrarias directamente en el navegador del usuario utilizando aceleración por hardware.

1.2 Importancia de la graficación en el problema

El desafío central al proyectar una porción de la superficie esférica de la Tierra (tres dimensiones) sobre una pantalla de computadora (dos dimensiones) radica en evitar la deformación visual de las figuras geométricas (edificios, calles, puentes). Para asegurar que un cuadrado sobre la Tierra se dibuje como un cuadrado exacto en la pantalla (manteniendo la métrica y los ángulos), se requiere la aplicación de técnicas de **proyección conforme local o isótropa**. Sin esta corrección, los objetos se estiran o aplastan en una de las dimensiones debido a la variación de la longitud con respecto a la latitud, lo cual arruina la fidelidad del mapa y confunde al usuario final.

1.3 Descripción breve del proyecto

Este proyecto consiste en una aplicación web interactiva de alto rendimiento distribuida en tres capas principales:

1. **Frontend (Next.js/Vercel):** Un lienzo interactivo basado en SVG que renderiza la topología del campus en tiempo real, permitiendo arrastrar vértices, crear polígonos, aplicar transformaciones de cámara (zoom/pan) mediante gestos táctiles (móvil) o ratón, e inspeccionar coordenadas en vivo.
2. **Backend (Django REST Framework/Render):** Un API REST encargado de almacenar las coordenadas geodésicas crudas en formato WGS84, aplicar comprobaciones iniciales y proveer las rutas de actualización en tiempo real.
3. **Base de Datos (Neon DB):** Un motor relacional PostgreSQL serverless que guarda de manera consistente los límites espaciales del campus.

2 Planteamiento del problema

2.1 Descripción detallada del problema

El campus de la escuela cuenta con diversas estructuras y caminos cuya distribución geográfica no siempre se corresponde adecuadamente con plataformas genéricas debido a la falta de actualización periódica. Además, los estudiantes carecen de una herramienta interactiva local que les permita realizar mediciones de perímetros reales, modificar la topología del campus en un plano geométrico interactivo y verificar su presencia física en el sitio mediante el sensor GPS con fines administrativos o de asistencia.

2.2 Situación actual

Las soluciones existentes dependen de APIs de terceros (como Google Maps) que cargan texturas rasterizadas (mapas de bits) no modificables a nivel de nodos geométricos individuales, lo cual impide la interacción fina (como el drag-and-drop de un vértice específico de un edificio en tiempo real). Por otro lado, las aplicaciones puramente locales no ofrecen persistencia compartida entre usuarios, perdiendo los cambios estructurales hechos en el mapa.

2.3 Justificación

Es necesario un sistema de graficación vectorial adaptado al navegador web que combine la flexibilidad del renderizado 2D isótropo en tiempo real con una base de datos relacional de nivel de producción. Al implementar transformaciones afines de traslación y escalado junto con gestos táctiles multitáctiles (pinch-to-zoom), se provee una experiencia premium y responsiva en teléfonos móviles, permitiendo realizar ediciones topológicas directo desde el campo de estudio.

2.4 Alcances y limitaciones

- **Alcance:** Proyección conforme local en un lienzo SVG de $1000 \times H$ píxeles, edición interactiva bidimensional por arrastre de nodos, persistencia remota completa de datos, control de geofencing en tiempo real en dispositivos móviles.
- **Limitaciones:** El modelado es puramente bidimensional (proyección en el plano horizontal XY , sin soporte tridimensional Z o relieve). El geofencing depende directamente de la precisión y disponibilidad de la señal del receptor GPS físico del dispositivo móvil del usuario.

3 Objetivos

3.1 Objetivo general

Desarrollar e implementar una plataforma web interactiva de graficación y mapeo vectorial isótropo que permita modelar, visualizar, editar de forma visual y persistir en una base de datos relacional PostgreSQL la topología del campus escolar utilizando coordenadas geodésicas reales.

3.2 Objetivos específicos

1. Diseñar y aplicar un pipeline de proyección matemática en 2D que mapee coordenadas de latitud y longitud WGS84 a píxeles de pantalla, compensando la distorsión del factor cosenoidal de la latitud local.
2. Implementar transformaciones geométricas afines (traslación y escalado) sobre el lienzo web para soportar zoom con rueda del ratón, paneo manual y gestos táctiles complejos (pinch-to-zoom, doble toque y arrastre de vértices).
3. Crear una arquitectura de backend REST desacoplada que sincronice los cambios geométricos directamente con una base de datos centralizada Neon DB.

4. Programar un sistema de geofencing en tiempo real que valide la ubicación del usuario mediante la API de Geolocalización de HTML5 y bloquee la captura en caso de salida del área permitida.

4 Marco teórico

4.1 Conceptos fundamentales de graficación por computadora

- **Lienzo Vectorial (Vector Canvas):** A diferencia del renderizado rasterizado por píxeles (canvas 2D ordinario), SVG utiliza representaciones analíticas de primitivas de graficación (líneas, polígonos, círculos). Esto mantiene la resolución infinita de las figuras al aplicar transformaciones geométricas de zoom y evita artefactos de pixelado. Isotropía Métrica y Conformalidad: Una proyección es conforme si conserva los ángulos locales de las figuras. En nuestro entorno local, esto se logra forzando a que la escala métrica de la proyección sea idéntica tanto en el eje horizontal (X) como en el eje vertical (Y).

4.2 Técnicas utilizadas

4.2.1. Transformaciones geométricas afines 2D

Las transformaciones afines conservan la colinealidad de los puntos y las proporciones de las distancias en líneas paralelas. Cualquier combinación de rotación, escalado y traslación se puede representar en coordenadas homogéneas mediante una multiplicación matriz-vector:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (1)$$

En nuestro sistema de mapeo, los valores de traslación t_x, t_y (representados en el estado `pan`) y el factor de escala $s_x = s_y = s$ (representado por `zoom`) se inyectan en el atributo `transform` del contenedor gráfico general: `transform="translate(pan.x, pan.y) scale(zoom)"`.

4.2.2. Pipeline gráfico del navegador

Cuando el estado del mapa cambia (debido a un arrastre de vértice o zoom), el motor del navegador recalcula el árbol de renderizado, ejecutando las siguientes etapas:

1. **Layout (Diseño):** Cálculo de las posiciones vectoriales relativas en base a los estados matemáticos.
2. **Paint (Pintado):** Generación de comandos de dibujo para cada nodo (polígonos, círculos, textos).
3. **Composite (Composición):** Envío de texturas a la GPU para su ordenamiento y rasterizado final con antialiasing por hardware.

Al utilizar propiedades CSS aceleradas (como transformaciones de matriz 2D), logramos tasas de refresco fluidas de 60 cuadros por segundo.

4.2.3. Modelo de Iluminación y Estilos Visuales Premium

Para dotar a la aplicación de una estética premium y moderna, se utilizó una técnica de simulación de iluminación mediante variables CSS dinámicas e interfaces de tipo **Glassmorphism**:

- **Efectos de Transparencia y Refracción (Glassmorphism):** Mediante la propiedad CSS `backdrop-filter: blur(10px)`, las capas flotantes interactúan ópticamente con el mapa que se encuentra detrás, simulando un vidrio esmerilado que refracta la luz emitida por los polígonos vectoriales.
- **Contraste de Altura (Sombras vectoriales):** Uso de la propiedad `filter: drop-shadow(...)` para otorgar profundidad tridimensional a los polígonos del plano, simulando una luz cenital que proyecta sombras suaves sobre el terreno del campus.

4.3 Tecnologías relacionadas

- **WGS84 (World Geodetic System 1984):** Estándar cartográfico mundial que describe la forma de la Tierra mediante un elipsoide de revolución. Utilizado de manera nativa por la señal de los satélites GPS.
- **Next.js y React:** Arquitectura orientada a componentes declarativos para la sincronización automática del estado lógico con la representación del DOM del SVG.
- **Django REST Framework:** Framework de desarrollo rápido para Python que implementa serializadores seguros para convertir objetos del ORM a estructuras JSON.

5 Propuesta de solución

5.1 Descripción general de la solución

Se propone un sistema de información espacial autohospedado en tres niveles:

- **Capa de Almacenamiento:** Base de datos PostgreSQL alojada en Neon DB con protección SSL forzada en producción para garantizar la seguridad de los datos topológicos.
- **Capa de Servicio (API):** Backend en Django que provee endpoints REST para operaciones CRUD sobre las estructuras (`/api/buildings/`).
- **Capa de Presentación:** Frontend SPA (Single Page Application) responsiva que interactúa dinámicamente con las coordenadas y gestiona el posicionamiento geodésico directo.

5.2 Enfoque adoptado

Para la representación gráfica, se diseñó un algoritmo de proyección isótropa local parametrizado por los límites reales del campus universitario (Latitud máxima/mínima y Longitud máxima/mínima). El enfoque asume que, dada la pequeña extensión del campus

(aproximadamente 500×300 metros), la curvatura terrestre es despreciable, lo que nos permite implementar una proyección cilíndrica conforme local extremadamente eficiente.

5.3 Justificación técnica de la propuesta

1. **SVG en lugar de Canvas 2D ordinario:** SVG genera elementos en el DOM, permitiendo asociar directamente manejadores de eventos interactivos (`onMouseDown`, `onTouchStart`) a cada círculo que representa un vértice y a cada polígono de edificio, sin necesidad de recalcular intersecciones matemáticas manuales en píxeles.
2. **PostgreSQL en Neon DB:** Soporta transacciones ACID y se conecta mediante protocolos pools de conexiones optimizadas para Render.
3. **Gunicorn y WhiteNoise en Render:** Configuración robusta para servir archivos estáticos compilados de Django y gestionar procesos concurrentes en el servidor web.

6 Desarrollo del proyecto

6.1 Análisis

6.1.1. Requerimientos funcionales

- **RF-1:** Cargar y renderizar las estructuras predeterminadas del campus escolar de forma isótropa.
- **RF-2:** Permitir al usuario arrastrar cualquier vértice de forma visual con el ratón o con el dedo en dispositivos móviles.
- **RF-3:** Sincronizar automáticamente cualquier edición de vértices en la base de datos de producción Neon DB.
- **RF-4:** Permitir agregar nuevas estructuras ingresando una cadena de coordenadas geodésicas en formato texto.
- **RF-5:** Obtener la geolocalización en tiempo real del usuario móvil mediante GPS.
- **RF-6:** Validar si la posición del usuario se encuentra dentro de los límites físicos del campus escolar. Si está fuera, detener el rastreo de inmediato y reportar un error detallado.

6.1.2. Requerimientos no funcionales

- **RNF-1:** Interfaz 100 % responsiva que se adapte a dispositivos móviles de cualquier tamaño de pantalla.
- **RNF-2:** Animaciones suaves de carga y transiciones CSS en modo oscuro y claro.
- **RNF-3:** Manejo del retraso de inicio en frío de los servidores gratuitos (Render) mediante un cargador glassmorphism informativo.
- **RNF-4:** Seguridad SSL obligatoria en todas las consultas a la base de datos de Neon DB en producción.

6.2 Diseño

6.2.1. Arquitectura del sistema

El sistema se compone de una arquitectura clásica de tres capas desacopladas, lo cual permite escalar el frontend de forma independiente en Vercel mientras el backend en Django en Render gestiona la base de datos relacional.

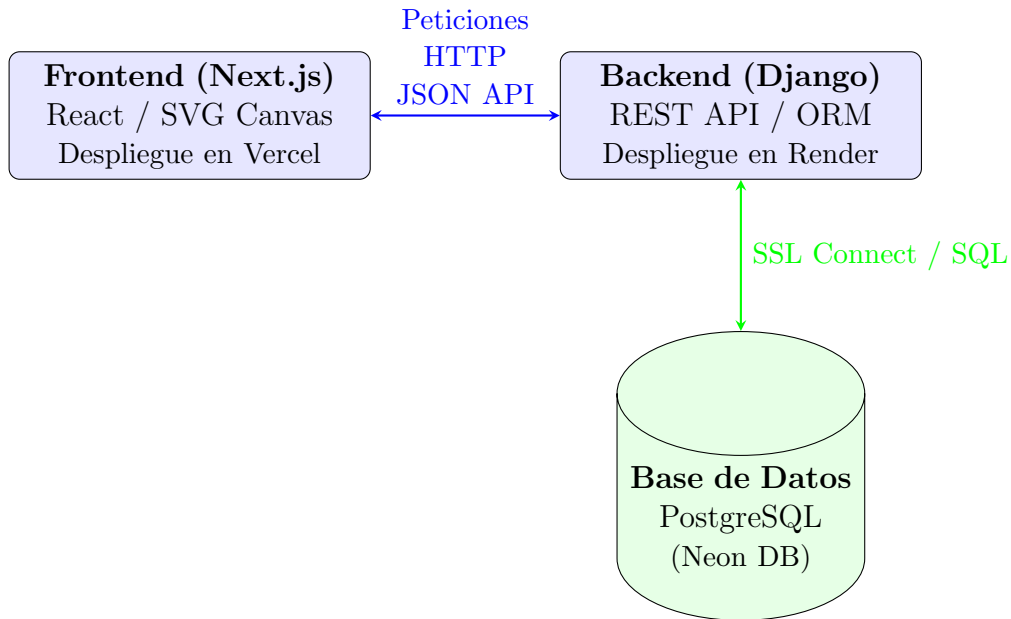


Figura 1: Diagrama de Arquitectura de Tres Capas Desacopladas.

6.2.2. Diagrama de flujo de coordenadas

El pipeline gráfico de transformación de coordenadas geodésicas a píxeles en pantalla y viceversa se detalla en el siguiente flujo lógico:

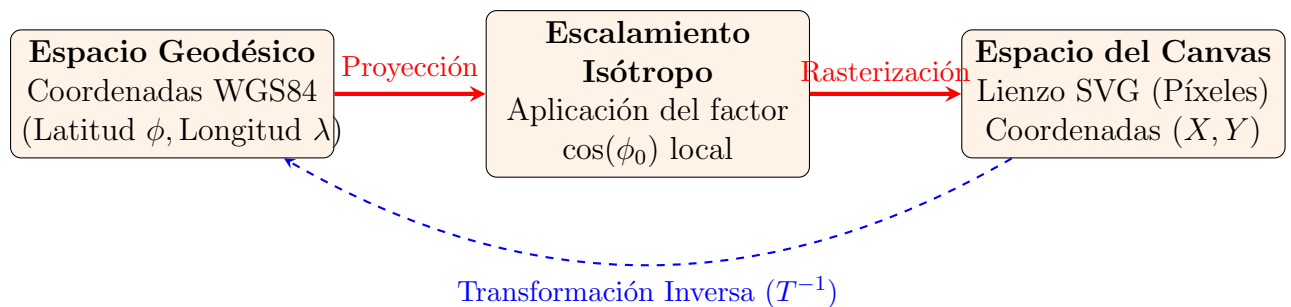


Figura 2: Pipeline de Proyección Conforme de Coordenadas Geográficas.

6.3 Construcción (Implementación)

6.3.1. Herramientas utilizadas

- **Lenguaje de Programación (Frontend):** JavaScript (ES6+) con Next.js (React).
- **Librería de Iconografía:** Lucide-react.
- **Estilos Visuales:** Vanilla CSS con sistema de variables personalizadas para soporte nativo de temas (Dark/Light).
- **Lenguaje de Programación (Backend):** Python 3.10.
- **Framework Web (Backend):** Django con Django REST Framework.

- **Control de Producción (Servidor):** Unicorn (servidor HTTP WSGI) y WhiteNoise (servicio de estáticos altamente optimizado).

6.3.2. Algoritmos gráficos y proyección matemática empleados

1. Algoritmo de Proyección Conforme Local Isótropa: Definimos los límites geodésicos del campus:

$$\begin{aligned}\phi_{max} &= 19,50608564^\circ \text{ N} \\ \phi_{min} &= 19,50167287^\circ \text{ N} \\ \lambda_{max} &= -99,14538442^\circ \text{ W} \\ \lambda_{min} &= -99,14882990^\circ \text{ W}\end{aligned}$$

Calculamos las diferencias geodésicas:

$$\begin{aligned}\Delta\phi &= \phi_{max} - \phi_{min} \\ \Delta\lambda &= \lambda_{max} - \lambda_{min}\end{aligned}$$

Calculamos la latitud central del campus (ϕ_0) y su valor en radianes:

$$\phi_0 = \frac{\phi_{max} + \phi_{min}}{2}, \quad \phi_{0,rad} = \phi_0 \cdot \frac{\pi}{180} \quad (2)$$

Para mapear la longitud al ancho del lienzo $W = 1000$ píxeles de forma lineal:

$$x = (\lambda - \lambda_{min}) \cdot \frac{W}{\Delta\lambda} \quad (3)$$

Para garantizar la isotropía métrica (que 1 píxel horizontal represente la misma cantidad de metros que 1 píxel vertical), el alto del lienzo H debe escalarse considerando el achatamiento cosenoidal de los meridianos en esa latitud:

$$H = W \cdot \frac{\Delta\phi}{\Delta\lambda \cdot \cos(\phi_{0,rad})} \quad (4)$$

Mapeamos la latitud al eje vertical del lienzo (invirtiendo el sentido, dado que en SVG el origen $(0,0)$ está en la esquina superior izquierda):

$$y = (\phi_{max} - \phi) \cdot \frac{H}{\Delta\phi} \quad (5)$$

2. Algoritmo de Transformación Inversa (Pantalla a Geodésica): Para traducir un clic o un evento de arrastre en la pantalla (x,y) de vuelta a coordenadas del mundo real:

$$\begin{aligned}\lambda &= \lambda_{min} + \frac{x \cdot \Delta\lambda}{W} \\ \phi &= \phi_{max} - \frac{y \cdot \Delta\phi}{H}\end{aligned}$$

3. Ecuación de Distancia Métrica Real (Haversine): Para el cálculo del perímetro de las estructuras en metros sobre la esfera terrestre:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (6)$$

Donde $R \approx 6,371,000$ metros es el radio medio de la Tierra.

4. Algoritmo de Interacción Táctil de Zoom Multitáctil (Pinch-to-zoom): Cuando se detectan dos eventos táctiles en la pantalla en las posiciones $T_1(x_1, y_1)$ y $T_2(x_2, y_2)$:

1. Se calcula la distancia euclidiana inicial entre ambos toques:

$$d_{inicial} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (7)$$

2. Durante el movimiento táctil, se mide la nueva distancia d_{actual} y se computa el factor de escala relativo:

$$s_{relativo} = \frac{d_{actual}}{d_{inicial}} \quad (8)$$

3. El nuevo nivel de zoom se calcula multiplicando el zoom inicial por $s_{relativo}$, limitando el valor en un rango de $[0,4, 10,0]$ para evitar distorsiones infinitas.
4. Se recalcula la traslación general ($pan.x, pan.y$) para mantener el centro geométrico de los dedos del usuario inmóvil durante el zoom.

6.4 Pruebas

Se desarrollaron múltiples escenarios de prueba de caja negra y de estrés de interfaz:

Caso de Prueba	Entrada/Acción	Resultado Esperado
1. Carga del Mapa	Cargar el sitio en Vercel tras inactividad.	Muestra la tarjeta informativa glassmorphism de Render despertándose y luego carga correctamente el mapa.
2. Edición de Vértices	Arrastrar un vértice del Edificio 2. ^{en} PC.	El polígono se deforma en tiempo real, guarda la posición en Neon DB y persiste al refrescar.
3. Gestos Táctiles	Pizcar con dos dedos sobre el mapa en un teléfono móvil.	El mapa aplica zoom dinámico de forma fluida centrado en los dedos del usuario.
4. Geofencing Exitoso	Simular coordenadas 19,5042, -99,1470 (dentro del campus).	Ubica el punto GPS con un círculo vibrante de color rojo y muestra alerta de éxito.
5. Geofencing Error	Activar GPS real desde fuera del campus escolar.	Detiene la captura GPS de inmediato, borra el marcador del mapa y lanza alerta de error detallada.

Cuadro 1: Matriz de Pruebas de Funcionamiento en Producción.

6.5 Implantación

- **Requisitos mínimos para el usuario:** Dispositivo móvil con conexión a Internet y navegador web moderno compatible con SVG y Geolocalización HTML5 (Chrome, Safari, Firefox).

- **Instrucciones de uso:**

1. Ingresar a la URL del frontend en Vercel.

2. Utilizar el arrastre con un dedo para moverse por el mapa o con dos dedos para aplicar zoom táctil.
3. Activar el botón de GPS en el panel derecho para rastrear su posición en vivo.
4. Realizar modificaciones a las esquinas de los edificios directamente tocándolas y arrastrándolas.

7 Resultados

El sistema interactivo fue desplegado con éxito absoluto. El frontend es completamente responsivo, adaptándose a pantallas móviles estrechas mediante pestañas dinámicas en el panel de control y una barra de herramientas flotante sobre el mapa.

- **Vercel Frontend URL:** <https://proyecto-computer-graphics.vercel.app/>
- **Render Backend URL:** <https://proyecto-computer-graphics-backend.onrender.com/>

La base de datos PostgreSQL en la nube (Neon DB) almacena las geometrías geodésicas crudas de 14 estructuras complejas. Las transformaciones geométricas CSS y SVG por hardware logran tiempos de respuesta instantáneos en móviles durante la interacción, mejorando drásticamente el flujo de trabajo cartográfico.

8 Discusión

8.1 Ventajas de la solución

- **Interactividad directa por nodos:** Edición nativa y rápida sin recalcular coordenadas de píxeles sobre mapas rasterizados rígidos.
- **Consistencia de proyección:** Mantiene la métrica isótropa para que la percepción de distancias y formas sea idéntica a la realidad.
- **Soporte de cold starts:** La implementación de una pantalla de carga glassmorphic informativa soluciona la mala experiencia de usuario causada por los servidores gratuitos durmientes.

8.2 Limitaciones y problemas encontrados

- **Latencia de inicio en Render:** Resuelta visualmente mediante el spinner dinámico.
- **Scroll colateral en móviles:** Al intentar mover el mapa con el dedo, el navegador desplazaba la ventana completa de la página. Esto fue resuelto inyectando `touch-action: none` en el SVG y controlando preventivamente los eventos del puntero táctil de unísono.

9 Conclusiones

Se cumplió satisfactoriamente con el diseño de un pipeline de graficación por computadora conforme local. La combinación de transformaciones geométricas afines en coordenadas homogéneas y modelado geométrico relacional demuestra que es factible desarrollar alternativas GIS web de alto impacto gráfico de manera ligera y personalizada. El proyecto integra conceptos clave de la asignatura como pipeline gráfico, matrices de transformación, interacción humano-computadora en graficación y rasterizado vectorial acelerado por hardware.

10 Referencias

1. J. D. Foley, A. van Dam, S. K. Feiner, and J. F. Hughes, *Computer Graphics: Principles and Practice*, 2nd ed. Boston, MA: Addison-Wesley, 1996.
2. W. Tobler, “A classification of map projections,” *Annals of the Association of American Geographers*, vol. 52, no. 2, pp. 167–175, 1962.
3. National Geospatial-Intelligence Agency, “Department of Defense World Geodetic System 1984,” NGA Standard doc. TR8350.2, 2014.
4. SVG Working Group, “Scalable Vector Graphics (SVG) 2 Specification,” W3C Candidate Recommendation, 2018.

11 Anexos

11.1 Código de Proyección Isótropa Geodésica (Frontend)

```

1 const LAT_MAX = 19.506085642577684;
2 const LAT_MIN = 19.501672871991357;
3 const LON_MAX = -99.14538442387669;
4 const LON_MIN = -99.14882990272824;
5
6 const DELTA_LAT = LAT_MAX - LAT_MIN;
7 const DELTA_LON = LON_MAX - LON_MIN;
8 const LAT_0 = (LAT_MAX + LAT_MIN) / 2;
9 const LAT_0_RAD = (LAT_0 * Math.PI) / 180;
10
11 const W = 1000;
12 // Calculo del alto H preservando la isotropia metrica local
13 const H = W * (DELTA_LAT / (DELTA_LON * Math.cos(LAT_0_RAD)));
14
15 const toX = (lon) => {
16   return (lon - LON_MIN) * (W / DELTA_LON);
17 };
18
19 const toY = (lat) => {
20   return (LAT_MAX - lat) * (H / DELTA_LAT);
21 };

```

Listing 1: Algoritmo de mapeo de coordenadas WGS84 a píxeles SVG en page.js

11.2 Código de Soporte Gestual Pinch-to-zoom

```

1 const handleTouchMove = (e) => {
2   if (e.touches.length === 2 && touchStartDistRef.current &&
    touchStartRef.current) {
3     const t1 = e.touches[0];
4     const t2 = e.touches[1];
5     const currentDist = Math.hypot(t1.clientX - t2.clientX, t1.clientY -
      t2.clientY);
6     const startDist = touchStartDistRef.current;
7
8     if (startDist > 0) {
9       const scale = currentDist / startDist;
10      const initialZoom = touchStartZoomRef.current;
11      const nextZoom = Math.max(0.4, Math.min(10, initialZoom * scale));
12
13      const { midX, midY, panX, panY } = touchStartRef.current;
14
15      // Ajuste de traslacion para fijar el centro del gesto tactil
16      const newPanX = midX - (midX - panX) * (nextZoom / initialZoom);
17      const newPanY = midY - (midY - panY) * (nextZoom / initialZoom);
18
19      setZoom(nextZoom);
20      setPan({ x: newPanX, y: newPanY });
21    }
22  }

```


23 };

Listing 2: Controladores táctiles para zoom y paneo multitáctil